

ESG RAG MCP — 專案啟動 Brief

版本 v0.1 · 討論稿

一、專案定位（一句話）

把 ESG 報告的下載、解析、向量化、檢索、citation 組裝這些複雜度收進後台，以 MCP server 形式對外提供，讓任何支援 MCP 的 agent（第一目標：Claude）可以直接查詢，不需要使用者自己處理原始報告。

二、目前資產盤點

可以繼續用的

資產	說明
Flask API 架構	已有可用的 Python web server 框架，MCP server 可以在此基礎上改寫或並行
RAGAS 評估框架	評估 retrieval 品質的方法論仍然有效，新 pipeline 跑完可以重新評估
領域理解	你已知道 ESG 語料的特性、查詢行為、chunk 邊界問題
225 次 ablation 的認識	雖然實驗結果不能直接沿用，但你知道要測什麼、怎麼測

要重做的

項目	原因
PDF 爬蟲	需要重新收集 50 家公司最新報告
Embedding pipeline	換成 Google embedding（可直接 input PDF），原本的向量不能用
Vector database	從 local ChromaDB 換成 local PostgreSQL + pgvector
Ablation 實驗	embedding model 換了，舊結果失效，需要重跑

三、技術決策清單

已決定

- Embedding model：Google embedding（支援直接 input PDF，省掉 PDF 解析步驟）

- **Vector database**：PostgreSQL + pgvector (local 先跑通，之後再考慮上雲)
- **架構型態**：MCP server (不做 UI，接 Claude 為第一目標)
- **開發策略**：循序漸進，pipeline → prototype → 再考慮產品化

待決定 (進入各階段前需要先釐清)

進 Phase 1 前要確認：

- ☐ Google embedding 具體用哪個 API？(text-embedding-004 還是 gemini-embedding-exp)？PDF 直接 input 的能力需要先確認哪個 endpoint 支援)
- ☐ 50 家公司的名單是哪 50 家？(台灣上市公司 ESG 報告，從哪個年份開始收？)
- ☐ PDF 從哪裡爬？(公開資訊觀測站 / 各公司官網 / 兩者並用？)

進 Phase 2 前要確認：

- ☐ chunk 策略：Google embedding 直接吃 PDF 的情況下，你的 chunk 單位是什麼？(整頁？段落？還是讓 Google 處理，你不切？)
- ☐ citation 的最小單位：你要回傳給 Claude 的是「頁碼 + 公司名」，還是「段落原文 + 來源」？

進 Phase 3 前要確認：

- ☐ MCP 要暴露哪些 tool？(至少需要想清楚 tool 名稱、input 參數、output 格式)
-

四、執行順序與各階段說明

整體邏輯：資料先行，API 次之，MCP 最後。每個階段都有一個「可以驗收的產出」，沒有達到就不進下一階段。

Phase 1：資料 Pipeline

目標：有一個 PostgreSQL + pgvector 資料庫，裡面存好 50 家公司的 ESG 報告向量，可以用 similarity search 查到相關段落。

要做的事：

1. 建立公司名單與來源清單

- 決定 50 家公司
- 找到每家公司 ESG 報告的下載 URL (可以先手動找，後面再考慮自動爬)
- 建立一個 CSV：公司名、年份、PDF URL、備注

2. 寫 PDF 下載腳本

- 批次下載 PDF，存到本機 data/raw/ 資料夾
- 記錄下載成功 / 失敗，失敗的要手動補

3. 設定 Google Embedding API

- 開 Google Cloud 帳號 (如果還沒有)

- 確認要用哪個 embedding endpoint
- 寫一個最小測試：送一個 PDF 頁面進去，確認回傳向量維度正確

4. 設定 PostgreSQL + pgvector (本機)

- 用 Docker 跑一個 local PostgreSQL
- 安裝 pgvector extension
- 建 table schema (至少要有：chunk_id、company、year、page、content、embedding)

5. 寫 ingestion pipeline

- 讀 PDF → 切分 (頁或段落) → 呼叫 Google embedding → 存進 pgvector
- 先用 1 家公司的 1 份報告跑通整條鏈，確認沒問題再跑全部

6. 跑完整 ingestion

- 把 50 家公司全部跑完
- 確認資料筆數合理 (幾千到幾萬筆 chunk)

驗收標準：

我可以在 Python 裡寫：

```
results = similarity_search("碳排放目標", top_k=5)
print(results)
```

然後看到 5 筆來自不同公司的相關段落，每筆都有公司名、頁碼、原文。

這個階段你需要學的東西：

- Docker 基本操作 (`docker run`、`docker-compose`)
- pgvector 的 table 建立與 similarity search 語法
- Google Cloud API 金鑰設定 (環境變數管理)
- Python 的 `psycopg2` 或 `asyncpg` (連接 PostgreSQL 的套件)

這個階段的風險：

- Google embedding 直接 input PDF 的 API 行為需要先小規模測試，確認它怎麼處理多頁 PDF (是整份送還是一頁一頁送)
- 部分公司的 PDF 可能是掃描版 (圖片)，Google embedding 能否處理需要確認
- 爬蟲部分：台灣公司官網結構不一，有些 PDF URL 可能會變，建議下載後存本機不要每次重爬

Phase 2：Retrieval 層 + 本機 API

目標：有一個可以在本機呼叫的 API endpoint，送進一個問題，回傳有 citation 的答案片段。

要做的事：

1. 寫 retrieval 函式

- input : query string
- 流程 : query → Google embedding → pgvector similarity search → 取 top-k chunks
- output : list of `{company, year, page, content, score}`

2. 加入 query 前處理 (可選, MVP 可跳過)

- 同義詞擴展 (「碳排放」 = 「溫室氣體」 = 「GHG」)
- 這塊可以先 hardcode 一個 mapping dict, 不需要用 LLM

3. 組裝 citation 格式

- 決定回傳格式, 例如 :

[台積電 2023 ESG Report, p.45]
「本公司承諾於 2030 年前達到...」

- 這個格式要讓 Claude 拿到後可以直接引用

4. 包成 Flask endpoint (可以用現有 Flask 架構)

- `POST /search` → 接 query → 回傳 chunks + citations
- 在本機跑, 用 curl 或 Postman 測試

5. 小規模品質評估

- 挑 10-20 個有代表性的問題, 手動看回傳結果是否合理
- 不需要重跑完整 RAGAS, 先用人工評估抓大方向

驗收標準 :

```
curl -X POST http://localhost:5000/search \  
-d '{"query": "台積電的碳中和目標是什麼"}'  
  
回傳 :  
{  
  "results": [  
    {  
      "company": "台積電",  
      "year": 2023,  
      "page": 45,  
      "content": "...",  
      "citation": "[台積電 2023 ESG Report, p.45]"  
    },  
    ...  
  ]  
}
```

這個階段你需要學的東西 :

- SQL 的向量搜尋語法 (pgvector 的 `<=>` operator)
- Python dict/list 的資料整理
- 基本的 Flask route 寫法 (你應該已經會了)

這個階段的風險：

- chunk 品質問題會在這裡第一次真正暴露 (太短沒有脈絡、太長噪音太多)
 - 如果 Google embedding 是整頁送的，而某頁有很多不相關內容，retrieval 精準度會下降
 - 這個階段如果評估結果很差，要回 Phase 1 調整 chunk 策略，這是正常的
-

Phase 3：MCP 包裝 + Prototype 驗證

目標：在 Claude Desktop 裡可以直接問 ESG 問題，Claude 會呼叫你的 MCP tool，回傳有 citation 的答案。

要做的事：

1. 設計 MCP tool schema

- 你需要決定暴露幾個 tool，每個 tool 的名稱、description、input 參數
- 最小版本：一個 tool 就夠

```
json
{
  "name": "search_esg_reports",
  "description": "搜尋台灣上市公司 ESG 報告的相關段落，  
回傳原文與來源引用。適合查詢公司的  
碳排放目標、ESG 政策、永續指標等。",
  "inputSchema": {
    "query": "string",
    "company": "string (optional)",
    "year": "integer (optional)"
  }
}
```

- **description 很重要**：這是給 Claude 看的，決定它什麼時候會呼叫這個 tool

2. 寫 MCP server

- 用 Python MCP SDK (`pip install mcp`)
- MCP server 內部呼叫你 Phase 2 做好的 retrieval 函式
- 本機跑，用 stdio transport (最簡單，不需要 HTTP server)

3. 設定 Claude Desktop 連接

- 在 Claude Desktop 的 config 加入你的 MCP server

- 測試：直接問 Claude 「台積電的碳中和目標是什麼」，看它會不會呼叫你的 tool

4. 手動測試 20-30 個問題

- 測試 tool 有沒有被正確觸發
- 測試 citation 格式 Claude 能不能正確引用
- 記錄失敗案例

5. 丟到 Facebook 社團詢問（你之前說的計畫）

- 這時候你有一個可以 demo 的東西了
- 可以錄一個 screen recording 給潛在使用者看
- 蒐集質性回饋：他們會問什麼問題？他們覺得什麼有用？什麼沒用？

驗收標準：

在 Claude Desktop 裡問：

「幫我整理台積電和聯發科在 2023 年的碳排放目標有什麼不同」

Claude 呼叫了 `search_esg_reports`，
回傳有引用來源的比較分析，
引用格式正確指向報告頁碼。

這個階段你需要學的東西：

- MCP Python SDK 的基本用法（官方文件有範例，不複雜）
- Claude Desktop 的 MCP config 設定方式（JSON 設定檔）
- stdio transport 是什麼（簡單說：MCP server 透過標準輸入輸出和 Claude 溝通，不需要開 HTTP port）

這個階段的風險：

- Tool description 寫不好，Claude 不知道什麼時候呼叫你的 tool
- 回傳的 chunk 太長，Claude context window 被塞滿
- 這個階段會發現 Phase 2 的 citation 格式有問題，需要來回調整

五、刻意不做的事（MVP 範圍外）

這些都是之後才需要考慮的，現在先不碰：

不做事	原因
上雲端部署	先在本機跑通，確認架構沒問題再說
後端 API 的安全機制（auth、rate limit）	MVP 不對外開放，不需要
支援使用者自傳 PDF	大幅增加複雜度，先鎖定固定語料
UI 介面	MCP 的本質就是讓 agent 當介面
多模型切換策略	先鎖定 Google embedding，穩定後再說
自動更新語料	手動維護就好，先驗證核心價值
完整 RAGAS 評估	Phase 2 先用人工評估，prototype 驗證後再決定要不要跑

六、開發環境建議

你不需要一開始就把環境設得很完整，但這幾件事值得在 Phase 1 開始前做好：

- 1. 用 `pyenv` 或 `conda` 管理 Python 版本，建一個獨立的虛擬環境給這個專案
- 2. 用 `.env` 檔管理 API 金鑰，不要把 key 寫在 code 裡
- 3. 用 Docker Compose 跑 PostgreSQL，這樣換電腦或之後上雲都容易
- 4. 用 Git 做版本控制，每個 phase 結束後打一個 tag

七、最需要先確認的一件事

在動手之前，先把 Google embedding 的 PDF input 能力小規模測試一遍。

原因：這是整個新 pipeline 的假設前提。如果 Google embedding 處理 PDF 的方式跟你預期的不一樣（例如它其實是把 PDF 轉成文字再 embed，而不是原生多模態 embed），你的 chunk 策略就要整個重想。

最小測試：

```
python

# 取一份 ESG 報告，送第一頁進去
# 確認：
# 1. API 怎麼接收 PDF (base64? file path? URL?)
# 2. 回傳的向量維度是多少
# 3. 整份 PDF 送 vs 一頁一頁送，行為有什麼不同
# 4. 費用大概多少（50 家公司全跑的成本估算）
```

這個測試做完，Phase 1 的其他工作才有明確的方向。

Brief 版本：v0.1 · 尚未進入執行 下次更新時機：Google embedding 小規模測試完成後